Bases de données et SQL

Support résumé — cours, exemples et mémo

SGBD : MySQL / MariaDB (XAMPP, phpMyAdmin)

Sommaire

Chapitre 1 — Notions de base de données
1.1 Donnée, information et base de données
1.2 Le système de gestion de base de données (SGBD)
1.3 Structure d'une base de données relationnelle

Chapitre 2 — Contraintes d'intégrité et représentations d'une BDR
2.1 Les contraintes d'intégrité
2.2 Représentations textuelle et graphique
2.3 Évaluation et correction d'une représentation

Chapitre 3 — Définition des données (LDD)
3.1 Les types de données
3.2 Créer et supprimer une base de données
3.3 Créer une table avec ses contraintes
3.4 Modifier une table
3.5 Supprimer une table

Chapitre 4 — Manipulation des données (LMD)
4.1 Insérer des enregistrements : INSERT
4.2 Modifier des enregistrements : UPDATE
4.3 Supprimer des enregistrements : DELETE

Chapitre 5 — Interrogation simple des données
5.1 La requête SELECT
5.2 Opérateurs de comparaison et opérateurs logiques
5.3 Les fonctions sur les dates

Chapitre 6 — Requêtes avancées
6.1 Fonctions d'agrégation, GROUP BY et HAVING
6.2 Requêtes sur plusieurs tables : les jointures
6.3 Requêtes imbriquées

Annexe — Importer et exporter au format SQL
A.1 Exporter avec phpMyAdmin
A.2 Contenu d'un fichier exporté
A.3 Importer avec phpMyAdmin

Mémo SQL



Chapitre 1 — Notions de base de données

1.1 Donnée, information et base de données

Notion de donnée

- Une donnée est une description d'une chose, d'un évènement, etc.
- Les données peuvent être conservées et classées (mémorisées) sous différentes formes : papier, numérique, texte, image, son…
- Un ensemble de données lié à un contexte donne une information.

Exemple : les données utilisées dans :

a. Un magasin qui vend des articles de sport :

- Nombre des articles
- Libellés des articles
- Quantité en stock pour chaque article
- Prix de vente de chaque article…

b. Une banque :

- Les services offerts par la banque
- Nombre des clients
- Les coordonnées de chaque client…

c. Le lycée :

- Les élèves (nom, prénom, classe)
- Les matières (libellé, coefficient)
- Les notes de chaque élève dans chaque matière

Remarque : La donnée « 14 » n'a pas de sens seule ; « Leila Ben Salah a obtenu 14 en Informatique » est une information.

Composants d'une information

1. Les données sont regroupées selon leur appartenance à un objet du monde réel (une personne, un fait, un objet…) appelé **entité**.
2. Chaque entité est décrite par un ensemble de propriétés appelées **attributs**.
3. Chaque attribut a une valeur.
4. L'ensemble des valeurs possibles d'un attribut est appelé **domaine** (exemple : une note est un réel de 0 à 20).
5. Une information provient de deux ou plusieurs entités reliées entre elles par une **association**.

Activité

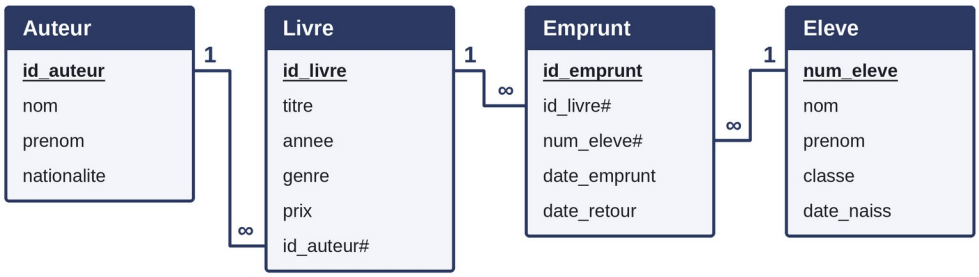
Dans une agence de location de voitures, dégager les données nécessaires pour chaque véhicule.

Solution : Les données nécessaires au commercial sont : immatriculation, modèle, catégorie, couleur, date de mise en circulation, km à la révision, km actuels, disponibilité, nom client, date de réservation, date de récupération…

Définition d'une base de données

Une base de données (BD), en anglais « database », est un ensemble de données structurées, non redondantes (non répétitives) et persistantes (non volatiles) relatives à un ou plusieurs domaines du monde réel.

Exemple 1 : la bibliothèque du lycée



Représentation graphique de la base « bibliotheque_lycee »

Un auteur écrit des livres ; un élève emprunte des livres ; chaque emprunt relie un élève et un livre à une date. Cette base sert d'exemple à tous les chapitres suivants.

Exemple 2 : Magasin

CLIENT

NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Eté	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

PRODUIT

NPRO	LIBELLE	PRIX	QSTOCK
CS262	CHEV. SAPIN 200x6x2	75	45
CS264	CHEV. SAPIN 200x6x4	120	2690
CS464	CHEV. SAPIN 400x6x4	220	450
PA45	POINTE ACIER 45 (2K)	105	580
PA60	POINTE ACIER 60 (1K)	95	134
PH222	PL. HETRE 200x20x2	230	782
PS222	PL. SAPIN 200x20x2	185	1220

DETAIL

NCOM	NPRO	QCOM
30178	CS464	25
30179	CS262	60
30179	PA60	20
30182	PA60	30
30184	CS464	120
30184	PA45	20
30185	CS464	260
30185	PA60	15
30185	PS222	600
30186	PA45	3
30188	CS464	180
30188	PA45	22
30188	PA60	70
30188	PH222	92

COMMANDE

NCOM	NCLI	DATECOM
30178	K111	21/12/2008
30179	C400	22/12/2008
30182	S127	23/12/2008
30184	C400	23/12/2008
30185	F011	2/01/2009
30186	C400	2/01/2009
30188	B512	3/01/2009

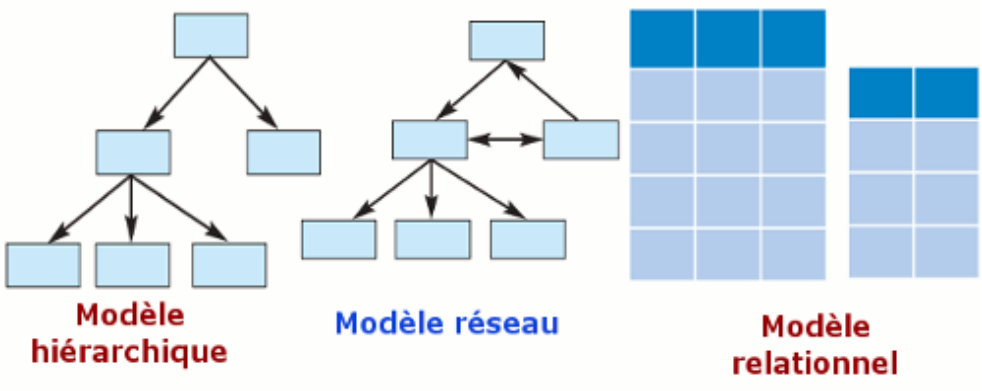
Utilité d'une base de données

- Centralisation des données : elle évite la redondance et permet l'utilisation des données par plusieurs programmes et plusieurs utilisateurs.
- Indépendance entre données et programmes.
- Intégration des liaisons entre les données.
- Partage : plusieurs utilisateurs consultent les mêmes données en même temps (exemple : le relevé de notes consulté par l'administration, les professeurs et les parents).
- Cohérence : une donnée modifiée une seule fois est à jour partout.

Modèles de bases de données

Il existe différents modèles de structuration de données qui se distinguent par la représentation des liens entre les données :

- Modèle hiérarchique
- Modèle réseau
- Modèle relationnel
- Modèle orienté objet



Dans ce cours, on étudie uniquement le modèle relationnel, le plus répandu.

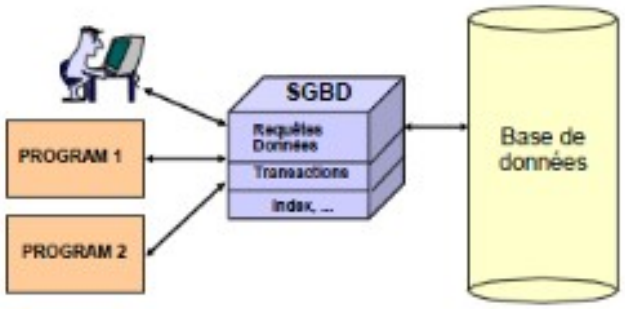
1.2 Le système de gestion de base de données (SGBD)

Définition

- Un SGBD (système de gestion de base de données) est un logiciel qui permet de créer, modifier, interroger et administrer une BD.
- Un SGBD est composé de deux parties :

a) le moteur, qui assure un ensemble de fonctions ;

b) l'interface, située entre l'utilisateur et le moteur, qui permet un accès facile et convivial à la BD.



Les fonctions d'un SGBD

Fonction	Rôle	Exemple au lycée
La définition des données	Créer les tables, leurs colonnes et leurs contraintes	Créer la table Eleve
La manipulation des données	Ajouter, modifier, supprimer et rechercher des données	Saisir les notes du trimestre
L'intégrité des données	Refuser les données qui violent les contraintes	Refuser une note de 25 sur 20
La gestion des accès concurrents	Permettre à plusieurs utilisateurs de travailler en même temps	Deux professeurs saisissent leurs notes simultanément
La confidentialité	Contrôler qui a le droit de voir ou de modifier	Un élève consulte ses notes sans pouvoir les modifier
La sécurité du fonctionnement	Sauvegarder et restaurer les données après une panne	Restaurer la base après une coupure de courant

Exemples de SGBD

Oracle, DB2, MySQL, PostgreSQL, Access, SQL Server, MariaDB (fourni avec XAMPP)…



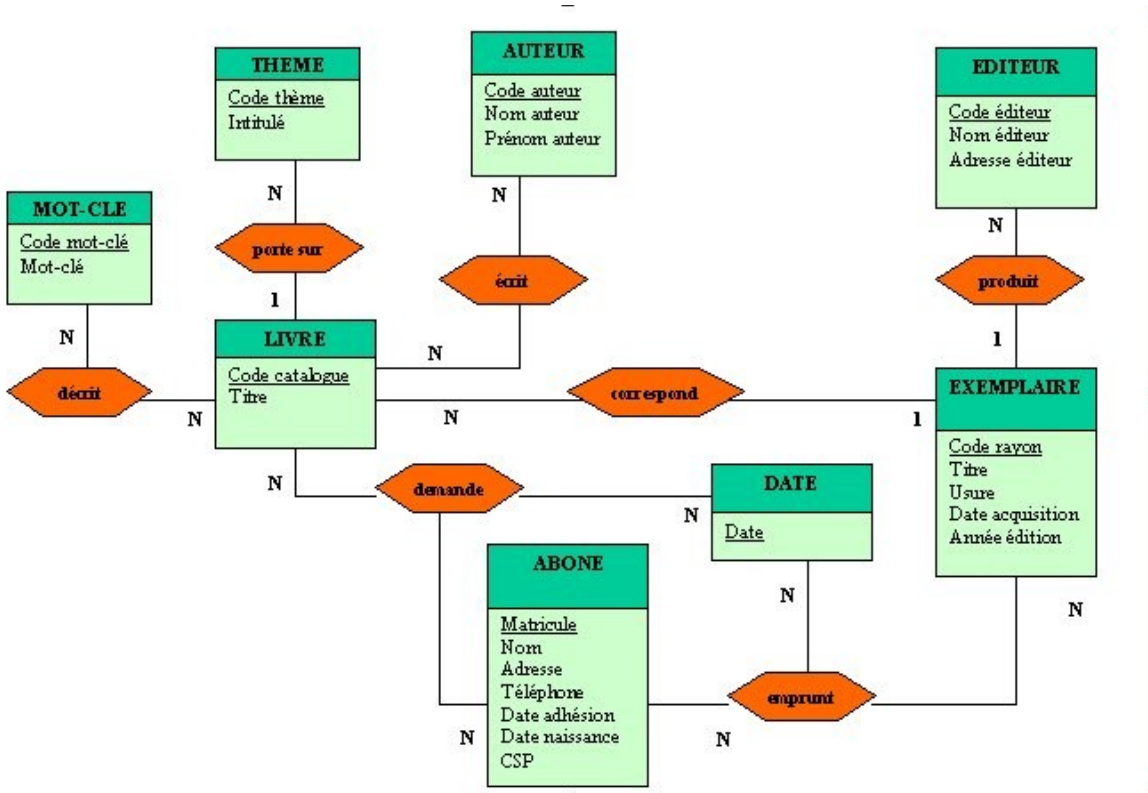
MongoDB, présent sur la figure, n'est pas un SGBD relationnel : c'est une base de données NoSQL, qui ne stocke pas ses données dans des tables.

1.3 Structure d'une base de données relationnelle

Définition

Le principe de base du modèle relationnel consiste à représenter aussi bien les objets que les liens à l'aide d'une structure appelée table.

Une table est une structure tabulaire dont les colonnes, appelées aussi attributs, correspondent aux caractéristiques de l'objet ou de l'association à représenter, et dont les lignes, appelées aussi tuples ou n-uplets, correspondent aux occurrences.



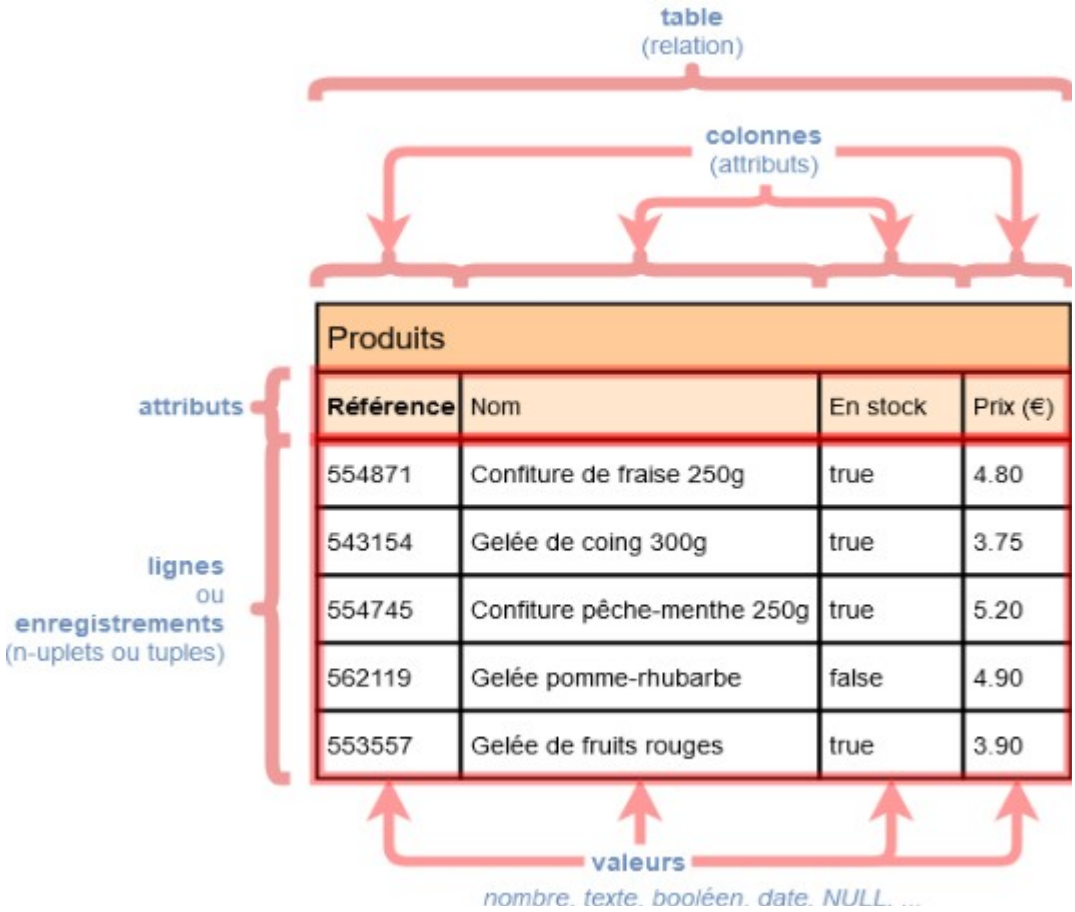
Le vocabulaire

Terme	Synonymes	Exemple (base bibliothèque)
Table	Relation	Livre
Colonne	Champ, attribut	titre
Ligne	Enregistrement, n-uplet, tuple	(3, 'L''Etranger', 1942, ...)
Clé primaire	Identifiant	id_livre
Clé étrangère	Référence vers une autre table	id_auteur# dans Livre

Composition d'une BDR

Selon le modèle relationnel, une BD est composée essentiellement de :

- Tables
- Colonnes
- Lignes
- Clés primaires
- Clés étrangères
- Contraintes d'intégrité



Produits

Référence	Nom	En stock	Prix (€)
554871	Confiture de fraise 250g	true	4.80
543154	Gelée de coing 300g	true	3.75
554745	Confiture pêche-menthe 250g	true	5.20
562119	Gelée pomme-rhubarbe	false	4.90
553557	Gelée de fruits rouges	true	3.90

Notion de table

Une table est un ensemble de données relatives à une entité et structurées sous forme de tableau.

Exemple : les données relatives aux articles commercialisés dans un magasin constituent la table « Article » suivante :

code_art	Désignation	Prix unitaire	Quantité en stock
V10	Vis 50*3	40	2500
V20	Vis 20*2	20	1300
B100	Boulon 90*15	450	100
C60	Clou 60*2	50	5000

Notion de colonne (champ, attribut)

Dans une table, une colonne correspond à une propriété élémentaire de l'objet décrit par cette table.

Une colonne est décrite par :

Élément	Description	Mot-clé SQL
Un nom	Comme le nom des variables dans les langages de programmation	—
Un type de données	Numérique, chaîne de caractères (texte), date, booléen (certains SGBD : multimédia)	INT, VARCHAR, DATE…
Une taille éventuelle	Longueur maximale (numérique ou texte)	VARCHAR(50)
Un indicateur de présence obligatoire	Obligatoire ou non (nulle : différent de zéro et de la chaîne vide)	NOT NULL
Une valeur par défaut éventuelle	Valeur prise quand aucune valeur n'est attribuée	DEFAULT
Une règle éventuelle	Valeurs autorisées (exemple : inférieures à une certaine valeur)	CHECK

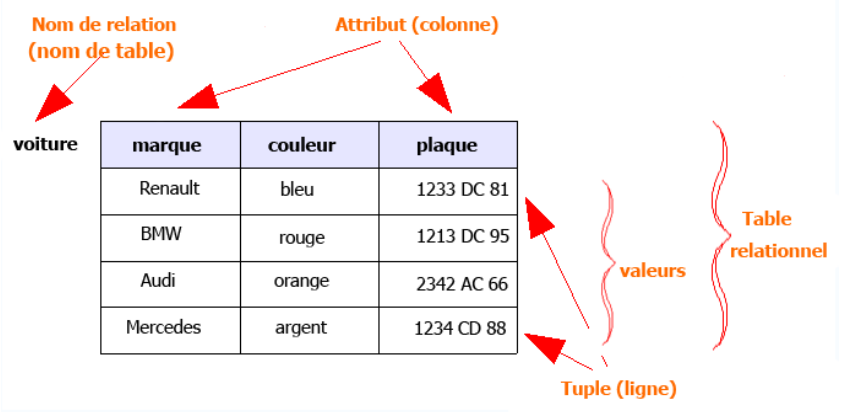
Exemple : la table « Article » :

Nom de la table : Article
Description : Détail des articles commercialisés

Nom colonne	Description	Type de données	Taille	Obligatoire	Valeur par défaut	Valeurs autorisées
Code_art	Code de l'article	Chaîne de caractères	20	Oui		
Des_art	Désignation de l'article	Chaîne de caractères	50	Oui		
PU	Prix unitaire de l'article	Numérique	8,3	Non		> 0
Qte_stok	Quantité en stock	Numérique	4	Non	0	>= 0

Notion de ligne (enregistrement, n-uplet)

Une ligne correspond à une occurrence de l'entité, du sujet représenté par la table.



Notion de clé primaire

Activité

1) Dans chacun des tableaux suivants, cocher chaque case permettant d'identifier d'une manière unique l'entité :

Citoyen	Nom	Prénom	Nom et Prénom	Age	N° CIN	N° Sécurité Sociale	Adresse

Voiture	Marque	Modèle	Marque et modèle	Age	N° Immatriculation	N° châssis	Lieu de fabrication

Etudiant	Nom	Prénom	Nom et Prénom	Age	N° CIN	N° Carte Etudiant	Adresse

Elève	Nom	Prénom	Nom et Prénom	Age	N° Téléphone	N° Inscription	Adresse

Ticket de transport	Prix	Ville de départ	Ville d'arrivée	Date	N° Ticket	Société de transport	Validité

- Une clé primaire est une colonne ou un ensemble de colonnes permettant d'identifier d'une façon unique chaque ligne de la table. Exemple : les lignes de la table Article sont identifiées par code_art.
- La clé primaire permet de connaître sans ambiguïté les valeurs des autres colonnes de la table.
- Chaque table d'une BDR a exactement une clé primaire, composée d'une ou de plusieurs colonnes.

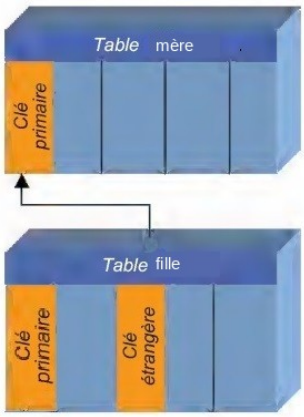
> **CONVENTION**
>
> La clé primaire est soulignée, dans la représentation textuelle comme dans la représentation graphique. Une clé primaire composée de plusieurs colonnes (clé composée) a toutes ses colonnes soulignées.

Notion de lien entre tables

Une base de données est la représentation d'un ou plusieurs domaines composés chacun d'un ensemble d'entités. Les différentes entités de chaque domaine sont généralement reliées par des liens (ou associations). Pour que la base de données constitue une représentation fidèle, les liens entre les sujets du monde réel doivent se retrouver dans la base de données.

Un lien entre deux tables A et B est représenté par l'ajout dans la table B d'une colonne correspondant à la clé primaire de la table A. Cette nouvelle colonne est une clé étrangère.

> **CONVENTION**
>
> La clé étrangère est suivie du symbole #. La table A, qui contient la clé primaire référencée, est la table mère ; la table B, qui contient la clé étrangère, est la table fille.



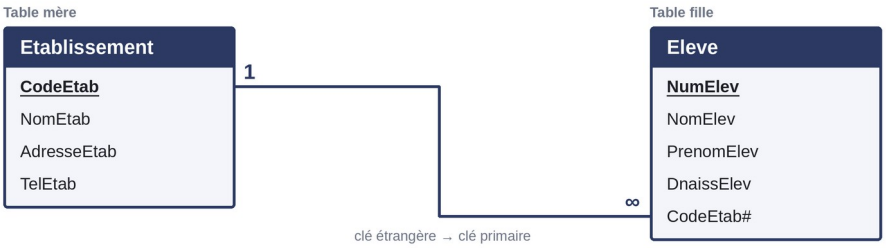
La clé étrangère de la table fille référence la clé primaire de la table mère

Exemple : les deux tables suivantes représentent les établissements et les élèves qui y sont inscrits :

Etablissement (CodeEtab, NomEtab, AdresseEtab, TelEtab)

Eleve (NumElev, NomElev, PrenomElev, DnaissElev, CodeEtab#)

Les deux tables peuvent être représentées sous la forme graphique suivante :

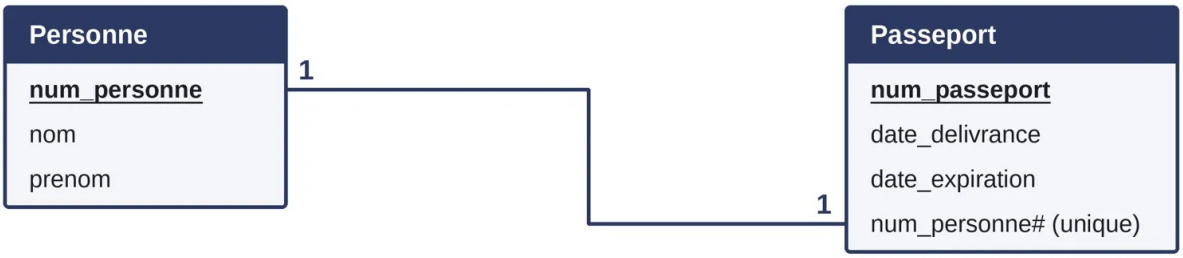


Etablissement est la table mère, Eleve la table fille : CodeEtab# référence CodeEtab

Types d'associations

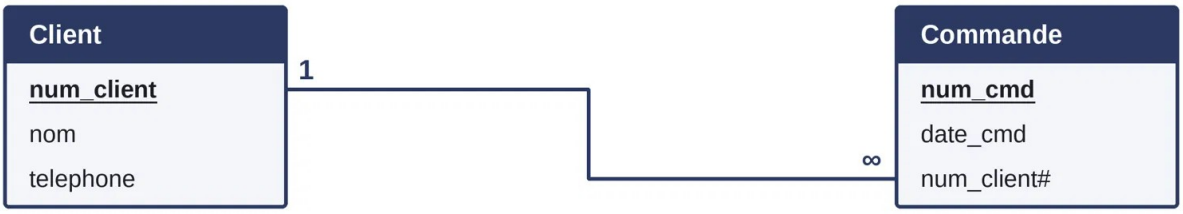
Association 1 à 1 (One-to-One)

- Un enregistrement dans A correspond à un seul enregistrement dans B.
- Exemple : une personne ↔ un passeport.



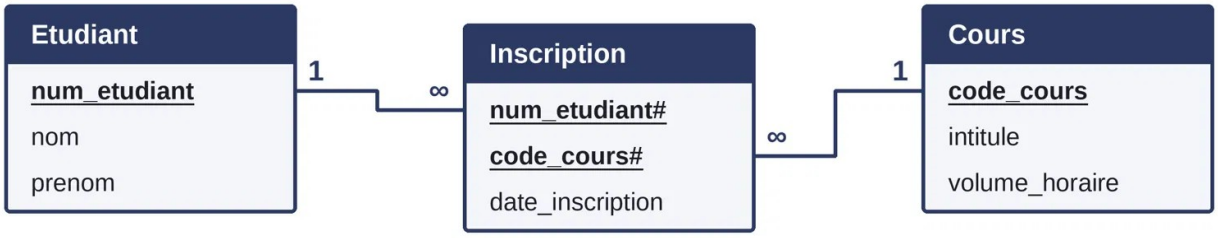
Association 1 à N (One-to-Many)

- Un enregistrement dans A correspond à plusieurs enregistrements dans B.
- Exemple : un client → plusieurs commandes.
- C'est le cas le plus fréquent.
- Traduction : la clé primaire de la table côté « 1 » (Client) devient clé étrangère dans la table côté « N » (Commande).



Association N à N (Many-to-Many)

- Plusieurs enregistrements dans A ↔ plusieurs enregistrements dans B.
- Exemple : étudiants ↔ cours.
- On utilise une table intermédiaire (table de jointure) qui contient les deux clés étrangères.



Exemple : un élève peut s'inscrire à plusieurs clubs, et un club accueille plusieurs élèves. La table intermédiaire Adhesion contient les deux clés étrangères, qui forment ensemble sa clé primaire.



Association N à N traduite par la table intermédiaire Adhesion

Eleve (num_eleve, nom, prenom)

Club (id_club, nom_club, jour)

Adhesion (num_eleve#, id_club#, date_adhesion)

Chapitre 2 — Contraintes d'intégrité et représentations d'une BDR

2.1 Les contraintes d'intégrité

Une contrainte d'intégrité est une règle imposée dans une base de données pour garantir que les données restent cohérentes, correctes et logiques. On distingue trois types de contraintes.

Contrainte de domaine

Règle : la valeur d'un champ doit respecter un type ou une condition.

Exemple : champ âge de type entier, avec la condition âge >= 18. Si on essaye de saisir âge = 15, la base refuse.

```
age INT CHECK (age >= 18)
```

Etudiant

id_etudiant	nom	age
1	Ben Ali	20
2	Trabelsi	19
3	Kefi	15

Insertion refusée
age = 15 ne respecte pas age >= 18

Contrainte de table

Règle : une colonne (ou un groupe de colonnes) sert de clé primaire ; une valeur doit être unique (UNIQUE).

Exemple : le champ email doit être unique.

```
id_etudiant INT PRIMARY KEY,  email VARCHAR(100) UNIQUE
```

Etudiant

id_etudiant	nom	email
1	Ben Ali	ali@ecole.tn
2	Trabelsi	karim@ecole.tn
2	Kefi	sami@ecole.tn
4	Gharbi	ali@ecole.tn

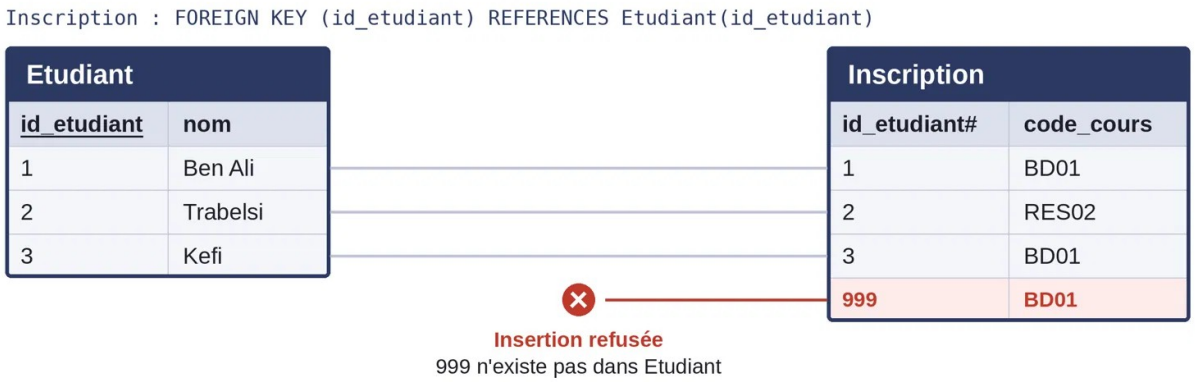
Refusée : id 2 existe déjà (clé primaire)

Refusée : email déjà utilisé (UNIQUE)

Contrainte référentielle

Règle : la valeur d'une clé étrangère doit exister comme clé primaire dans la table référencée.

Exemple : Inscription fait référence à Etudiant via id_etudiant. Si id_etudiant = 999 n'existe pas dans Etudiant, l'insertion dans Inscription est refusée.



Résumé : une contrainte d'intégrité est une règle qui empêche la base de stocker des données incohérentes (âge négatif, email en doublon, étudiant qui n'existe pas).

Correspondance avec les mots-clés SQL

Type de contrainte	Mots-clés SQL	Exemple
Domaine	Type de la colonne, NOT NULL, DEFAULT, CHECK	note DECIMAL(4,2) CHECK (note BETWEEN 0 AND 20)
Table	PRIMARY KEY, UNIQUE	email VARCHAR(100) UNIQUE
Référentielle	FOREIGN KEY ... REFERENCES, ON DELETE CASCADE, ON UPDATE CASCADE	FOREIGN KEY (id_auteur) REFERENCES Auteur(id_auteur)

NOT NULL porte sur les valeurs d'une colonne : on le classe avec les contraintes de domaine.

Les contraintes en action

Le SGBD refuse toute donnée qui viole une contrainte. Exemples sur la base bibliothèque :

Tentative	Contrainte violée
Ajouter un livre avec id_livre = 1 (déjà utilisé)	Table : clé primaire
Ajouter un livre dont l'auteur n° 99 n'existe pas	Référentielle
Ajouter un livre avec un prix de -5	Domaine : CHECK (prix > 0)
Supprimer un auteur qui a écrit des livres	Référentielle (table mère)

Exemple : la requête suivante est refusée, car l'auteur n° 99 n'existe pas dans la table Auteur (contrainte référentielle) :

```
INSERT INTO Livre VALUES (9, 'Notre-Dame de Paris', 1831, 'Roman', 15, 99);
```

À RETENIR

- Domaine : chaque valeur respecte le type et les conditions de sa colonne.
- Table : chaque ligne est identifiée par une clé primaire unique et renseignée.
- Référentielle : chaque clé étrangère référence une clé primaire existante.

2.2 Représentations textuelle et graphique

La structure d'une base de données relationnelle se décrit de deux façons équivalentes : par du texte ou par un schéma. Il faut savoir passer de l'une à l'autre, puis évaluer une représentation et la corriger.

Représentation textuelle

Elle consiste à décrire les tables, les colonnes et les liens entre les tables en utilisant du texte :

Nom_table (colonne1, colonne2, ..., colonneK#, ...)

colonne1, soulignée, est la clé primaire ; colonneK#, suivie de #, est une clé étrangère.

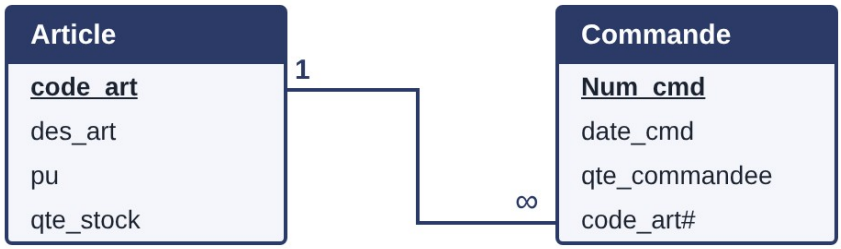
Exemple :

Article (code_art, des_art, pu, qte_stock)

Commande (Num_cmd, date_cmd, qte_commandee, code_art#)

Représentation graphique

Elle consiste à décrire les tables, les colonnes et les liens entre les tables en utilisant des symboles graphiques.



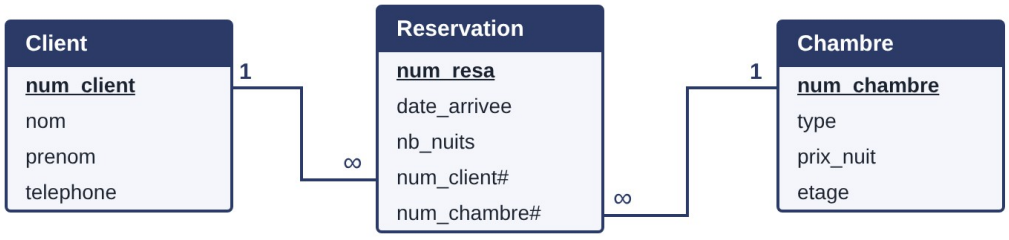
Représentation graphique des tables Article et Commande

Lecture : chaque table est un rectangle ; la clé primaire est soulignée ; le lien relie la clé étrangère (côté ∞, table fille) à la clé primaire qu'elle référence (côté 1, table mère).

Convertir une représentation en une autre

Graphique → textuelle	Textuelle → graphique
1. Une ligne Nom_table (...) par rectangle.	1. Un rectangle par table, titré par son nom.
2. Les colonnes dans l'ordre, la clé primaire soulignée.	2. Une ligne par colonne, la clé primaire soulignée.
3. Chaque colonne reliée côté ∞ est une clé étrangère : ajouter #.	3. Pour chaque colonne X#, tracer un lien vers la clé primaire de la table référencée.
4. Vérifier que chaque # correspond à un lien.	4. Écrire 1 côté table mère, ∞ côté table fille.

Exemple : la réservation d'hôtel



Représentation graphique à convertir

Représentation textuelle obtenue :

Client (num_client, nom, prenom, telephone)

Chambre (num_chambre, type, prix_nuit, etage)

Reservation (num_resa, date_arrivee, nb_nuits, num_client#, num_chambre#)

Exemple : la base bibliothèque

La représentation graphique du chapitre 1 s'écrit :

Auteur (id_auteur, nom, prenom, nationalite)

Livre (id_livre, titre, annee, genre, prix, id_auteur#)

Eleve (num_eleve, nom, prenom, classe, date_naiss)

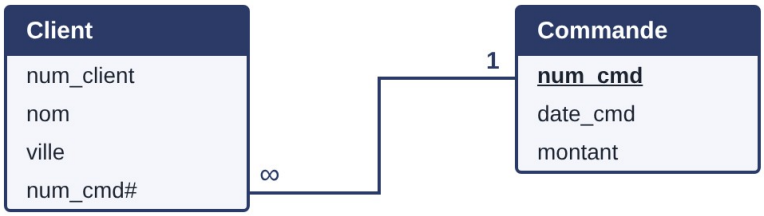
Emprunt (id_emprunt, id_livre#, num_eleve#, date_emprunt, date_retour)

2.3 Évaluation et correction d'une représentation

Évaluer une représentation, c'est vérifier qu'elle respecte les contraintes d'intégrité et la convention. Liste de contrôle :

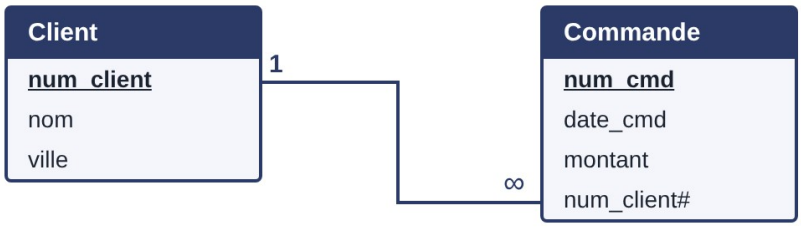
N°	Vérification	Anomalie typique
1	Chaque table a une clé primaire, soulignée	Aucune colonne soulignée
2	La clé primaire identifie vraiment chaque ligne	Clé primaire = nom (deux élèves peuvent avoir le même nom)
3	Chaque clé étrangère est suivie de # et liée à une clé primaire	Lien vers une colonne qui n'est pas une clé primaire
4	Association 1 à N : la clé étrangère est du côté N	num_commande# placé dans Client
5	Association N à N : une table intermédiaire	Clés étrangères croisées entre les deux tables
6	Pas de redondance ni de colonne contenant une liste	Nom du client recopié dans chaque commande ; colonne « liste_produits »

Cas 1 — Clients et commandes



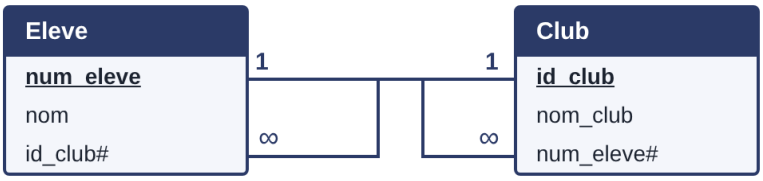
Représentation erronée

- Anomalie 1 : la table Client n'a pas de clé primaire (num_client n'est pas souligné).
- Anomalie 2 : la clé étrangère est du mauvais côté. Un client passe plusieurs commandes : avec num_cmd# dans Client, un client ne pourrait avoir qu'une commande.



Correction : num_client souligné, clé étrangère num_client# dans Commande

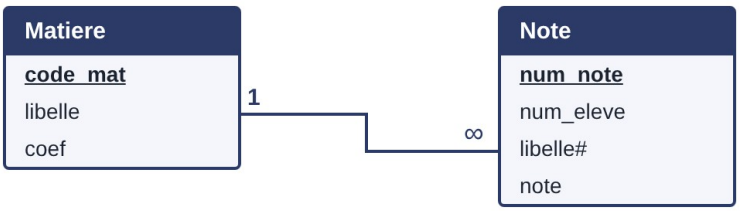
Cas 2 — Élèves et clubs



Représentation erronée

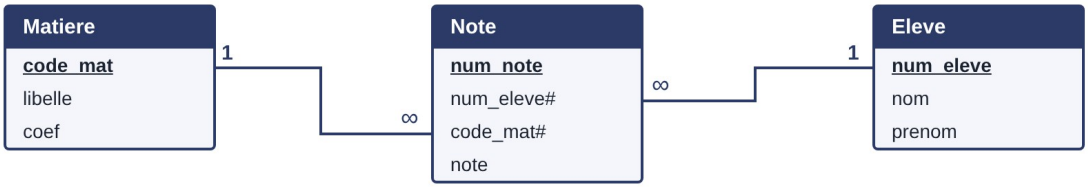
- Anomalie : l'association est N à N (un élève, plusieurs clubs ; un club, plusieurs élèves), traduite par deux clés étrangères croisées. Chaque élève ne pourrait appartenir qu'à un club, et chaque club ne retiendrait qu'un élève.
- Correction : supprimer id_club# de Eleve et num_eleve# de Club, et créer la table intermédiaire Adhesion (chapitre 1).

Cas 3 — Matières et notes



Représentation erronée

- Anomalie 1 : libelle# référence libelle, qui n'est pas la clé primaire de Matiere : une clé étrangère doit référencer une clé primaire (code_mat).
- Anomalie 2 : num_eleve n'est pas suivi de # et aucune table Eleve n'existe : la contrainte référentielle ne peut pas être vérifiée.



Correction : code_mat# et num_eleve# référencent les clés primaires de Matiere et Eleve

Cas 4 — Une représentation textuelle erronée

Commande (num_cmd, date_cmd, nom_client, tel_client, liste_produits)

- Pas de clé primaire soulignée.
- Redondance : nom et téléphone du client recopiés dans chaque commande ; une table Client s'impose.
- Colonne multivaluée : liste_produits contient plusieurs valeurs ; une commande contient plusieurs produits et un produit figure dans plusieurs commandes (N à N) : table intermédiaire Ligne_commande.

Correction :

Client (num_client, nom_client, tel_client)

Produit (code_prod, designation, prix)

Commande (num_cmd, date_cmd, num_client#)

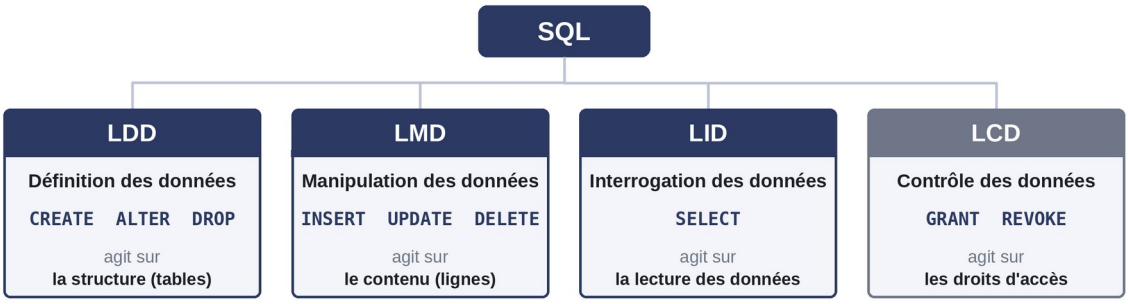
Ligne_commande (num_cmd#, code_prod#, quantite)

À RETENIR

- Clé primaire soulignée, clé étrangère suivie de #.
- Association 1 à N : la clé étrangère va dans la table côté N.
- Association N à N : une table intermédiaire dont la clé primaire est composée des deux clés étrangères.
- Une représentation correcte n'a ni redondance ni colonne contenant une liste.

Chapitre 3 — Définition des données (LDD)

Le langage SQL (Structured Query Language) est le langage standard utilisé pour interagir avec les bases de données relationnelles. Il permet de créer, modifier et interroger les bases de données de manière structurée et cohérente. Le SQL est divisé en plusieurs sous-langages, chacun ayant des fonctions spécifiques.



Les quatre sous-langages de SQL

Sous-langage	Rôle	Commandes	Chapitres
LDD (Langage de Définition de Données)	Crée, modifie, supprime les tables et leur structure	CREATE, ALTER, DROP	3
LMD (Langage de Manipulation de Données)	Ajoute, modifie, supprime le contenu des tables	INSERT, UPDATE, DELETE	4
LID (Langage d'Interrogation de Données)	Lit et extrait les informations	SELECT	5 et 6
LCD (Langage de Contrôle de Données)	Gère les autorisations d'accès des utilisateurs	GRANT, REVOKE	Non étudié

Le Langage de Définition de Données (LDD), ou DDL (Data Definition Language), est utilisé pour définir et gérer les structures de la base de données, telles que les tables. Les principales commandes LDD sont :

- CREATE : créer des objets de base de données (base, tables...) ;
- ALTER : modifier la structure des objets existants ;
- DROP : supprimer des objets de la base de données.

Exemple de création de table :

```
CREATE TABLE produits (
id INT PRIMARY KEY,
nom VARCHAR(100),
prix DECIMAL(10, 2)
);
```

Exemple de modification de table :

```
ALTER TABLE produits ADD COLUMN stock INT;
```

Exemple de suppression de table :

```
DROP TABLE produits;
```

3.1 Les types de données

Type	Signification	Exemple
INT	Entier	num_eleve INT
DECIMAL(p, d)	Réel : p chiffres dont d après la virgule	prix DECIMAL(6,3) : 25.500
CHAR(n)	Chaîne de longueur fixe	code_postal CHAR(4)
VARCHAR(n)	Chaîne de longueur variable, n caractères au maximum	nom VARCHAR(50)
TEXT	Chaîne de longueur variable (texte long)	resume TEXT
DATE	Date (AAAA-MM-JJ)	'2026-10-13'
TIME	Temps (HH:MM:SS)	'08:30:00'
DATETIME	Date et heure	'2026-10-13 08:30:00'

TYPES PROPRES À MYSQL / MARIADB

Pour une année, utiliser INT avec une contrainte CHECK : le type YEAR de MySQL et MariaDB refuse les années antérieures à 1901 (l'année 1862 est rejetée).

Pour une liste de valeurs autorisées, utiliser VARCHAR avec CHECK (colonne IN (...)) plutôt que le type ENUM, propre à MySQL.

AUTO_INCREMENT, utilisé dans certains exemples, numérote automatiquement la clé primaire : c'est une facilité de MySQL et MariaDB, hors du standard SQL.

3.2 Créer et supprimer une base de données

Créer une base

```
CREATE DATABASE nom_base;
```

Supprimer une base

```
DROP DATABASE nom_base;
```

Exemple : supprimer la base bibliothèque

```
DROP DATABASE bibliotheque_lycee;
```

> **ATTENTION**
> DROP est définitif : la structure et toutes les données disparaissent. Exporter la base avant (annexe).

3.3 Créer une table avec ses contraintes

Créer une table

```
CREATE TABLE nom_table (
col1 TYPE [NOT NULL] [PRIMARY KEY],
col2 TYPE [DEFAULT valeur],
CONSTRAINT nom UNIQUE(colonne),
FOREIGN KEY (col_etrangere) REFERENCES table_mere(col_pere)
);
```

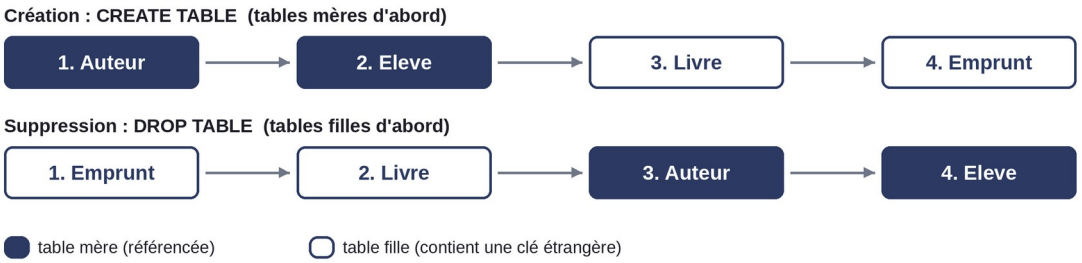
Les contraintes dans CREATE TABLE

Une contrainte s'écrit après la colonne qu'elle concerne (contrainte de colonne) ou à la fin de la table, éventuellement nommée avec CONSTRAINT (contrainte de table). Une clé primaire composée ou une clé étrangère se déclarent à la fin.

Contrainte	Rôle	Écriture
NOT NULL	Interdit une valeur nulle	nom VARCHAR(50) NOT NULL
DEFAULT	Valeur par défaut	nationalite VARCHAR(30) DEFAULT 'Inconnue'
CHECK	Condition à respecter	CHECK (prix > 0)
PRIMARY KEY	Clé primaire	id_livre INT PRIMARY KEY
UNIQUE	Valeurs toutes différentes	isbn VARCHAR(13) UNIQUE
FOREIGN KEY … REFERENCES	Clé étrangère	FOREIGN KEY (id_auteur) REFERENCES Auteur(id_auteur)
ON DELETE CASCADE	Supprime les lignes filles avec la ligne mère	… REFERENCES Eleve(num_eleve) ON DELETE CASCADE
ON UPDATE CASCADE	Répercute la modification de la clé primaire	… REFERENCES Auteur(id_auteur) ON UPDATE CASCADE

Exemple : créer la base bibliothèque

Les tables mères (Auteur, Eleve) sont créées avant les tables filles (Livre, Emprunt), car une clé étrangère doit référencer une table existante.



Ordre de création et de suppression des tables de la base bibliothèque

```
CREATE DATABASE bibliotheque_lycee;
USE bibliotheque_lycee;

CREATE TABLE Auteur (
  id_auteur   INT PRIMARY KEY,
  nom         VARCHAR(50) NOT NULL,
  prenom      VARCHAR(50),
  nationalite VARCHAR(30) DEFAULT 'Inconnue'
);

CREATE TABLE Eleve (
  num_eleve  INT PRIMARY KEY,
  nom        VARCHAR(50) NOT NULL,
  prenom     VARCHAR(50) NOT NULL,
  classe     VARCHAR(10) NOT NULL,
  date_naiss DATE
);

CREATE TABLE Livre (
  id_livre  INT PRIMARY KEY,
  titre     VARCHAR(100) NOT NULL,
  annee     INT CHECK (annee BETWEEN 1500 AND 2100),
  genre     VARCHAR(20),
  prix      DECIMAL(6,3) CHECK (prix > 0),
  id_auteur INT,
  CONSTRAINT fk_livre_auteur FOREIGN KEY (id_auteur) REFERENCES Auteur(id_auteur)
);

CREATE TABLE Emprunt (
  id_emprunt   INT PRIMARY KEY,
  id_livre     INT NOT NULL,
  num_eleve    INT NOT NULL,
  date_emprunt DATE NOT NULL,
  date_retour  DATE,
  CONSTRAINT fk_emprunt_livre FOREIGN KEY (id_livre) REFERENCES Livre(id_livre),
  CONSTRAINT fk_emprunt_eleve FOREIGN KEY (num_eleve) REFERENCES Eleve(num_eleve)
    ON DELETE CASCADE,
  CONSTRAINT chk_dates CHECK (date_retour >= date_emprunt)
);
```

ON DELETE CASCADE sur Emprunt : si un élève quitte le lycée et qu'on le supprime, ses emprunts sont supprimés avec lui. Sans cette option, la suppression serait refusée.

3.4 Modifier une table

Modifier une table

```
ALTER TABLE nom_table
ADD colonne TYPE,
DROP COLUMN colonne,
MODIFY colonne TYPE,
ADD CONSTRAINT nom CHECK(condition),
DROP CONSTRAINT nom;
```

Les modifications ci-dessous partent de la base bibliothèque.

Ajouter une colonne

```
ALTER TABLE Eleve ADD telephone VARCHAR(8);
```

Supprimer une colonne

```
ALTER TABLE Eleve DROP COLUMN date_naiss;
```

Le mot COLUMN est facultatif : ALTER TABLE Eleve DROP date_naiss; est équivalent.

Modifier le type d'une colonne

```
ALTER TABLE Eleve MODIFY nom VARCHAR(80) NOT NULL;
```

MODIFY redéfinit entièrement la colonne : il faut répéter NOT NULL, sinon la contrainte est perdue.

> **ATTENTION**
>
> L'écriture ALTER TABLE table1 ALTER colonne type existe dans certains SGBD, mais MariaDB et MySQL (XAMPP) la refusent.
>
> Avec XAMPP, on utilise MODIFY.

Renommer une colonne

```
ALTER TABLE Eleve RENAME COLUMN classe TO niveau;
ALTER TABLE Eleve CHANGE niveau classe VARCHAR(10) NOT NULL;
```

RENAME COLUMN renomme ; CHANGE renomme et redéfinit le type en une seule instruction.

> **ATTENTION**
>
> L'écriture ALTER TABLE table1 RENAME nom1 TO nom2, sans le mot COLUMN, est refusée par MariaDB.
>
> Écrire RENAME COLUMN, ou utiliser CHANGE.

Ajouter une contrainte

```
ALTER TABLE Eleve ADD CONSTRAINT uq_eleve UNIQUE (nom, prenom, date_naiss);
ALTER TABLE Livre ADD CONSTRAINT chk_genre CHECK (genre IN ('Roman', 'Poésie', 'Théâtre'));
```

Supprimer une contrainte

```
ALTER TABLE Livre DROP CONSTRAINT fk_livre_auteur;
```

D'où l'intérêt de nommer les contraintes avec CONSTRAINT : on les retrouve par leur nom.

Activer et désactiver une contrainte

En SQL standard (et dans Oracle), ALTER TABLE table1 DISABLE CONSTRAINT nom désactive une contrainte et ENABLE CONSTRAINT la réactive. MariaDB et MySQL ne connaissent pas cette instruction.

Avec XAMPP, on désactive temporairement la vérification des clés étrangères, puis on la réactive :

```
SET FOREIGN_KEY_CHECKS = 0;
-- opérations qui violeraient temporairement les clés étrangères
SET FOREIGN_KEY_CHECKS = 1;
```

C'est ce que fait phpMyAdmin pendant une importation (annexe).

3.5 Supprimer une table

Supprimer une table

```
DROP TABLE nom_table;
```

```
DROP TABLE Emprunt;
DROP TABLE Livre;
```

Une table mère ne peut pas être supprimée tant qu'une table fille la référence :

```
DROP TABLE Auteur;
```

On supprime donc les tables filles d'abord (Emprunt, puis Livre), puis les tables mères.

> **À RETENIR**
>
> - CREATE DATABASE / CREATE TABLE : créer ; les tables mères avant les tables filles.
> - ALTER TABLE … ADD, DROP, MODIFY, CHANGE, RENAME COLUMN, ADD CONSTRAINT, DROP CONSTRAINT : modifier.
> - DROP TABLE / DROP DATABASE : supprimer ; les tables filles avant les tables mères.

Chapitre 4 — Manipulation des données (LMD)

Le Langage de Manipulation de Données (LMD) ajoute, modifie et supprime les lignes des tables. La structure des tables ne change pas.

Ajouter (INSERT)

```
INSERT INTO table (col1, col2) VALUES (val1, val2);
INSERT INTO table VALUES (val1, val2, ...);
```

Modifier (UPDATE)

```
UPDATE table SET col1 = val1 WHERE condition;
```

Supprimer (DELETE)

```
DELETE FROM table WHERE condition;
```

À comprendre : les trois commandes du LMD agissent sur les lignes (enregistrements) d'une table ; sa structure ne change pas. Avant d'enregistrer une ligne, le SGBD vérifie toutes les contraintes d'intégrité : si une seule est violée, la ligne entière est refusée.

Bonne pratique : avant un UPDATE ou un DELETE, exécuter d'abord un SELECT avec la même clause WHERE : il affiche exactement les lignes qui seront modifiées ou supprimées.

4.1 Insérer des enregistrements : INSERT

```
INSERT INTO Eleve VALUES (107, 'Ayari', 'Nour', '4SI1', '2008-12-01');
INSERT INTO Eleve (num_eleve, nom, prenom, classe) VALUES (108, 'Sassi', 'Ines', '3SI1');
INSERT INTO Auteur (id_auteur, nom) VALUES (7, 'Tlili');
```

- Sans liste de colonnes, il faut donner une valeur pour chaque colonne, dans l'ordre de la table.
- Avec une liste, les colonnes omises prennent leur valeur par défaut (DEFAULT) ou NULL : la nationalité de l'auteur 7 vaut « Inconnue », la date de naissance d'Ines vaut NULL.
- Textes et dates entre apostrophes ; nombres sans apostrophes ; séparateur décimal : le point.
- Une apostrophe dans un texte se double : 'L''Etranger'.

Insérer plusieurs lignes : on peut insérer plusieurs lignes en une seule requête, en séparant les groupes de valeurs par des virgules :

```
INSERT INTO Auteur (id_auteur, nom, prenom, nationalite) VALUES
  (8, 'Mahfouz', 'Naguib', 'Egypte'),
  (9, 'Darwich', 'Mahmoud', 'Palestine');
```

Ordre des insertions : une ligne fille ne peut être insérée que si la ligne mère qu'elle référence existe déjà. On insère donc l'auteur n° 8 avant son livre :

```
INSERT INTO Livre VALUES (9, 'Impasse des deux palais', 1956, 'Roman', 14.500, 8);
```

Les contraintes refusent les données incohérentes

Clé primaire en double (contrainte de table)

```
INSERT INTO Eleve VALUES (101, 'Kefi', 'Ali', '4SI1', '2008-02-02');
```

Réponse du SGBD : ERROR 1062 (23000): Duplicate entry '101' for key 'PRIMARY'

Explication : l'élève n° 101 existe déjà : deux lignes ne peuvent pas avoir la même clé primaire.

Valeur obligatoire absente (NOT NULL, sans valeur par défaut)

```
INSERT INTO Eleve (num_eleve, nom, prenom) VALUES (109, 'Kefi', 'Ali');
```

Réponse du SGBD : ERROR 1364 (HY000): Field 'classe' doesn't have a default value

Explication : la colonne classe est NOT NULL et n'a pas de valeur par défaut : elle doit être renseignée.

Condition non respectée (CHECK, contrainte de domaine)

```
INSERT INTO Livre VALUES (9, 'Nedjma', 1956, 'Roman', -5, 1);
```

Réponse du SGBD : ERROR 4025 (23000): CONSTRAINT `Livre.prix` failed for `bibliotheque_lycee`.`Livre`

Explication : le prix -5 ne respecte pas la condition CHECK (prix > 0).

Livre inexistant (contrainte référentielle)

```
INSERT INTO Emprunt VALUES (9, 99, 101, '2026-10-13', NULL);
```

Réponse du SGBD : ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (… CONSTRAINT `fk_emprunt_livre` …)

Explication : aucun livre n'a le numéro 99 : la clé étrangère id_livre doit référencer un livre existant.

Code d'erreur	Signification	Contrainte
1062	Valeur déjà utilisée (doublon)	Table : PRIMARY KEY, UNIQUE
1364	Valeur obligatoire manquante	Domaine : NOT NULL
4025	Condition non respectée	Domaine : CHECK
1452	Ligne mère inexistante	Référentielle : FOREIGN KEY
1451	Ligne mère encore référencée	Référentielle : FOREIGN KEY

4.2 Modifier des enregistrements : UPDATE

```
-- Leila rend « L'Etranger » le 13 octobre
UPDATE Emprunt SET date_retour = '2026-10-13' WHERE id_emprunt = 2;
-- Changement de classe : les élèves de 3SI1 passent en 4SI1
UPDATE Eleve SET classe = '4SI1' WHERE classe = '3SI1';
-- Hausse de 10 % des prix des romans
UPDATE Livre SET prix = prix * 1.10 WHERE genre = 'Roman';
```

Exemple détaillé : la hausse de 10 % des romans. Avant l'UPDATE :

titre	prix
Les Misérables	25.500
L'Etranger	9.800
La Peste	11.200
Le Livre des jours	13.500

après l'UPDATE, chaque prix est multiplié par 1,10 et arrondi à 3 décimales, comme le prévoit le type DECIMAL(6,3) :

titre	prix
Les Misérables	28.050
L'Etranger	10.780
La Peste	12.320
Le Livre des jours	14.850

Modifier plusieurs colonnes à la fois : les affectations sont séparées par des virgules après SET :

```
UPDATE Eleve SET classe = '4SI2', date_naiss = '2008-12-02' WHERE num_eleve = 108;
```

> **ATTENTION**
>
> Sans WHERE, UPDATE modifie toutes les lignes de la table. Vérifier d'abord la condition avec un SELECT.

Une mise à jour qui violerait une contrainte est refusée, comme une insertion :

```
UPDATE Emprunt SET date_retour = '2026-09-01' WHERE id_emprunt = 1;
```

Réponse du SGBD : ERROR 4025 (23000): CONSTRAINT `chk_dates` failed for `bibliotheque_lycee`.`Emprunt`

Explication : l'emprunt n° 1 date du 15 septembre : une date de retour au 1er septembre violerait la contrainte chk_dates (date_retour >= date_emprunt).

4.3 Supprimer des enregistrements : DELETE

```
DELETE FROM Emprunt WHERE date_retour IS NOT NULL AND date_retour < '2026-10-01';
```

Vérifier avant de supprimer : le SELECT ci-dessous, avec la même condition, montre les lignes que ce DELETE supprime :

```
SELECT id_emprunt, date_retour FROM Emprunt WHERE date_retour IS NOT NULL AND date_retour < '2026-10-01';
```

Résultat :

id_emprunt	date_retour
1	2026-09-29
3	2026-09-27
5	2026-09-24

Supprimer une ligne mère référencée est refusé…

```
DELETE FROM Auteur WHERE id_auteur = 1;
```

Réponse du SGBD : ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (… CONSTRAINT `fk_livre_auteur` …)

Explication : Victor Hugo (auteur n° 1) a écrit deux livres de la table Livre : le supprimer laisserait deux livres sans auteur.

…sauf si la clé étrangère est déclarée ON DELETE CASCADE. Emprunt → Eleve l'est : supprimer l'élève 101 supprime aussi ses deux emprunts.

```
DELETE FROM Eleve WHERE num_eleve = 101;
```



ON DELETE CASCADE : supprimer l'élève 101 supprime ses deux emprunts

Commande	Ce qui disparaît	Ce qui reste
DELETE FROM t WHERE condition;	Les lignes qui vérifient la condition	Les autres lignes et la table
DELETE FROM t;	Toutes les lignes	La table vide (sa structure)
DROP TABLE t;	La table : structure et données	Rien

ATTENTION

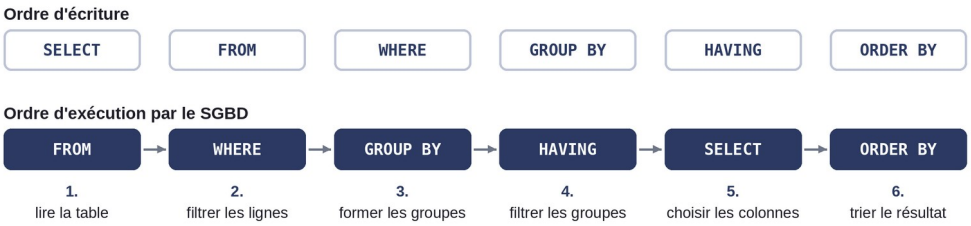
DELETE FROM Emprunt; sans WHERE vide la table (la structure reste). DROP TABLE supprime la table elle-même.

Chapitre 5 — Interrogation simple des données

5.1 La requête SELECT

```
SELECT [DISTINCT] expression [, col, ... ou *] [[AS] alias]
FROM table1 [[AS] alias] [, table2, ...]
[WHERE condition]
[GROUP BY critère]
[HAVING condition]
[ORDER BY expression [ASC/DESC]]
```

Les clauses s'écrivent toujours dans cet ordre. Seules SELECT et FROM sont obligatoires.



Ordre d'écriture et ordre d'exécution d'une requête SELECT

Le SGBD n'exécute pas les clauses dans l'ordre où on les écrit : il lit d'abord la table (FROM), filtre les lignes (WHERE), forme les groupes, puis choisit les colonnes. C'est pourquoi un alias défini dans SELECT ne peut pas être utilisé dans WHERE.

Sélectionner des colonnes

Toutes les colonnes, toutes les lignes

```
SELECT * FROM Auteur;
```

Résultat :

id_auteur	nom	prenom	nationalite
1	Hugo	Victor	France
2	Camus	Albert	France
3	Chebbi	Abou el Kacem	Tunisie
4	Messadi	Mahmoud	Tunisie
5	Gibran	Khalil	Liban
6	Hussein	Taha	Egypte

Certaines colonnes, un alias, un tri

```
SELECT titre, annee AS parution FROM Livre ORDER BY annee DESC;
```

Explication : AS parution renomme la colonne annee dans le résultat ; DESC trie du plus grand au plus petit. Sans précision, le tri est croissant (ASC).

Résultat :

titre	parution
Les Chants de la vie	1955
Le Barrage	1955
La Peste	1947
L'Etranger	1942
Le Livre des jours	1929
Le Prophète	1923
Les Misérables	1862
Les Contemplations	1856

Sans doublons : DISTINCT

```
SELECT DISTINCT genre FROM Livre;
```

Résultat :

genre
Roman
Poésie
Théâtre

Explication : sans DISTINCT, la requête renverrait 8 lignes, une par livre, avec des genres répétés.

Une expression calculée

```
SELECT titre, prix, prix * 0.8 AS prix_solde FROM Livre WHERE genre = 'Poésie';
```

Résultat :

titre	prix	prix_solde
Les Contemplations	12.000	9.6000
Les Chants de la vie	8.500	6.8000
Le Prophète	7.900	6.3200

Explication : prix_solde n'existe pas dans la table : la colonne est calculée pour chaque ligne au moment de la requête, puis nommée par l'alias.

MySQL et MariaDB acceptent aussi LIMIT n en fin de requête pour n'afficher que les n premières lignes ; cette clause ne fait pas partie du standard SQL.

Trier sur plusieurs colonnes

Principe : les lignes sont triées par classe ; à classe égale, elles sont triées par nom.

```
SELECT nom, prenom, classe FROM Eleve ORDER BY classe, nom;
```

Résultat :

nom	prenom	classe
Haddad	Sami	3SI1
Mansour	Yasmine	3SI1
Ben Salah	Leila	4SI1
Trabelsi	Karim	4SI1
Gharbi	Amel	4SI2
Jaziri	Omar	4SI2

5.2 Opérateurs de comparaison et opérateurs logiques

Opérateur	Exemple
= <> > < >= <=	annee >= 1950
AND, OR, NOT	genre = 'Roman' AND prix < 12
IN (liste)	nationalite IN ('Tunisie', 'Liban')
BETWEEN a AND b	annee BETWEEN 1900 AND 1950 (bornes comprises)
LIKE motif	titre LIKE 'Le%' (% : toute suite de caractères ; _ : un caractère)
IS NULL, IS NOT NULL	date_retour IS NULL

Comparaison et AND

```
SELECT titre, prix FROM Livre WHERE genre = 'Roman' AND prix < 12;
```

Résultat :

titre	prix
L'Etranger	9.800
La Peste	11.200

Explication : avec AND, une ligne est retenue seulement si les deux conditions sont vraies.

IN

```
SELECT nom, prenom FROM Auteur WHERE nationalite IN ('Tunisie', 'Liban');
```

Résultat :

nom	prenom
Chebbi	Abou el Kacem
Messadi	Mahmoud
Gibran	Khalil

Explication : nationalite IN ('Tunisie', 'Liban') équivaut à nationalite = 'Tunisie' OR nationalite = 'Liban'. NOT IN retient les valeurs absentes de la liste.

BETWEEN

```
SELECT titre, annee FROM Livre WHERE annee BETWEEN 1900 AND 1950;
```

Résultat :

titre	annee
L'Etranger	1942
La Peste	1947
Le Prophète	1923
Le Livre des jours	1929

Explication : BETWEEN 1900 AND 1950 équivaut à annee >= 1900 AND annee <= 1950 : les deux bornes sont comprises.

LIKE

```
SELECT nom, prenom, classe FROM Eleve WHERE nom LIKE 'B%' OR nom LIKE '_a%';
```

Résultat :

nom	prenom	classe
Ben Salah	Leila	4SI1
Mansour	Yasmine	3SI1
Jaziri	Omar	4SI2
Haddad	Sami	3SI1

Lire un motif LIKE : % remplace une suite quelconque de caractères (même vide) ; _ remplace exactement un caractère.

Motif	Signification	Exemple qui correspond
'Le%'	commence par Le	Le Barrage
'%s'	se termine par s	Les Misérables
'%de%'	contient de	Les Chants de la vie
'_a%'	a en 2e lettre	Mansour, Jaziri, Haddad

IS NULL : les livres non rendus

```
SELECT id_emprunt, id_livre, num_eleve FROM Emprunt WHERE date_retour IS NULL;
```

Résultat :

id_emprunt	id_livre	num_eleve
2	3	101
6	6	105
7	1	103

> **ATTENTION**
>
> On n'écrit jamais date_retour = NULL : la comparaison avec NULL ne donne jamais vrai. Il faut IS NULL.

Priorité de AND et de OR

Principe : AND est évalué avant OR, comme la multiplication avant l'addition. Les deux requêtes suivantes ne donnent pas le même résultat.

```
SELECT titre, genre, prix FROM Livre
WHERE genre = 'Roman' OR genre = 'Poésie' AND prix < 10;
```

Résultat :

titre	genre	prix
Les Misérables	Roman	25.500
L'Etranger	Roman	9.800
La Peste	Roman	11.200
Les Chants de la vie	Poésie	8.500
Le Prophète	Poésie	7.900
Le Livre des jours	Roman	13.500

Explication : la requête a été lue comme genre = 'Roman' OR (genre = 'Poésie' AND prix < 10) : tous les romans sont affichés, même les plus chers.

```
SELECT titre, genre, prix FROM Livre
WHERE (genre = 'Roman' OR genre = 'Poésie') AND prix < 10;
```

Résultat :

titre	genre	prix
L'Etranger	Roman	9.800
Les Chants de la vie	Poésie	8.500
Le Prophète	Poésie	7.900

À retenir : en cas de doute, mettre des parenthèses.

5.3 Les fonctions sur les dates

Fonction	Rôle
DATE(ch)	Extrait la date d'une valeur DATE ou DATETIME
DAY(d), MONTH(d), YEAR(d)	Jour, mois, année d'une date
NOW()	Date et heure courantes
ADDDATE(d, n)	Ajoute n jours (ou un intervalle) à une date
DATEDIFF(d1, d2)	Nombre de jours entre d2 et d1 (d1 - d2)

À comprendre : une date s'écrit entre apostrophes au format 'AAAA-MM-JJ'. DATEDIFF(d1, d2) calcule d1 − d2 : placer la date la plus récente en premier pour obtenir un nombre positif.

Élèves nés en 2008

```
SELECT nom, prenom, date_naiss FROM Eleve WHERE YEAR(date_naiss) = 2008;
```

Résultat :

nom	prenom	date_naiss
Ben Salah	Leila	2008-03-14
Trabelsi	Karim	2008-11-02
Jaziri	Omar	2008-05-09

Durée de chaque emprunt rendu, en jours

```
SELECT id_emprunt, date_emprunt, date_retour, DATEDIFF(date_retour, date_emprunt) AS
nb_jours FROM Emprunt WHERE date_retour IS NOT NULL;
```

Résultat :

id_emprunt	date_emprunt	date_retour	nb_jours
1	2026-09-15	2026-09-29	14
3	2026-09-20	2026-09-27	7
4	2026-09-22	2026-10-08	16
5	2026-09-10	2026-09-24	14
8	2026-10-02	2026-10-09	7

Date de retour prévue : 15 jours après l'emprunt

```
SELECT id_emprunt, date_emprunt, ADDDATE(date_emprunt, 15) AS retour_prevu FROM Emprunt
WHERE date_retour IS NULL;
```

Résultat :

id_emprunt	date_emprunt	retour_prevu
2	2026-10-01	2026-10-16
6	2026-10-05	2026-10-20
7	2026-10-06	2026-10-21

Emprunts en retard aujourd'hui (plus de 15 jours, non rendus)

```
SELECT id_emprunt, num_eleve, DATEDIFF(NOW(), date_emprunt) AS jours FROM Emprunt WHERE
date_retour IS NULL AND DATEDIFF(NOW(), date_emprunt) > 15;
```

Le résultat dépend du jour où la requête est exécutée (NOW()).

Extraire le mois et l'année

```
SELECT id_emprunt, date_emprunt FROM Emprunt
WHERE MONTH(date_emprunt) = 9 AND YEAR(date_emprunt) = 2026;
```

Résultat :

id_emprunt	date_emprunt
1	2026-09-15
3	2026-09-20
4	2026-09-22
5	2026-09-10

Explication : les emprunts de septembre 2026. MONTH() seul ne suffit pas : il retiendrait aussi septembre des autres années.

Chapitre 6 — Requêtes avancées

6.1 Fonctions d'agrégation, GROUP BY et HAVING

Les fonctions d'agrégation

Les fonctions d'agrégation servent à faire des calculs sur les lignes recherchées. Elles ne s'utilisent qu'avec la commande SELECT.

Fonction	Rôle	Exemple
COUNT([DISTINCT] X)	Compter le nombre de lignes du résultat obtenu par la commande SELECT.	SELECT COUNT(*) FROM client; affiche le nombre de clients.
SUM(X)	Calculer la somme des valeurs de l'attribut X.	SELECT SUM(ChiffresAffaires) FROM client;
MIN(X)	Calculer le minimum des valeurs de l'attribut X.	SELECT MIN(ChiffresAffaires) FROM client;
MAX(X)	Calculer le maximum des valeurs de l'attribut X.	SELECT MAX(ChiffresAffaires) FROM client;
AVG(X)	Calculer la moyenne des valeurs de l'attribut X.	SELECT AVG(ChiffresAffaires) FROM client;

Sur la base bibliothèque

```
SELECT COUNT(*) AS nb_livres, MIN(prix) AS moins_cher, MAX(prix) AS plus_cher, AVG(prix) AS prix_moyen, SUM(prix) AS valeur_fonds FROM Livre;
```

Résultat :

nb_livres	moins_cher	plus_cher	prix_moyen	valeur_fonds
8	7.900	25.500	12.3000000	98.400

Explication : sans GROUP BY, une fonction d'agrégation résume toutes les lignes en une seule ligne de résultat.

COUNT(*), COUNT(colonne), COUNT(DISTINCT colonne)

```
SELECT COUNT(*) AS emprunts, COUNT(date_retour) AS rendus, COUNT(DISTINCT num_eleve) AS emprunteurs FROM Emprunt;
```

Résultat :

emprunts	rendus	emprunteurs
8	5	5

COUNT(colonne) ignore les valeurs NULL : 5 emprunts rendus sur 8.

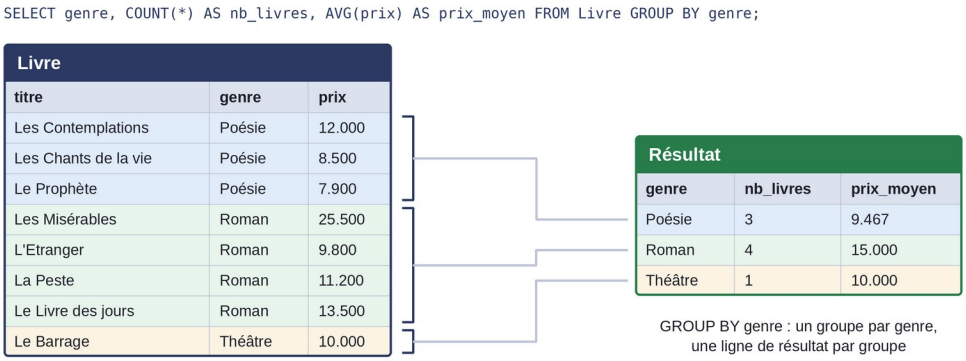
Écriture	Ce qui est compté	Valeur
COUNT(*)	Toutes les lignes	8
COUNT(date_retour)	Les lignes où date_retour n'est pas NULL	5
COUNT(DISTINCT num_eleve)	Les valeurs différentes de num_eleve	5

Grouper : GROUP BY

GROUP BY forme un groupe par valeur du critère ; la fonction d'agrégation s'applique à chaque groupe.

Nombre de livres et prix moyen par genre

```
SELECT genre, COUNT(*) AS nb_livres, AVG(prix) AS prix_moyen FROM Livre GROUP BY genre;
```



GROUP BY genre : un groupe par genre, une ligne de résultat par groupe

Nombre d'emprunts par élève

```
SELECT num_eleve, COUNT(*) AS nb_emprunts FROM Emprunt GROUP BY num_eleve ORDER BY
nb_emprunts DESC;
```

Résultat :

num_eleve	nb_emprunts
101	2
102	2
103	2
104	1
105	1

Trier sur un agrégat : on peut trier sur le résultat d'un agrégat en utilisant son alias dans ORDER BY. Ici, à égalité, l'ordre des élèves n'est pas garanti : ajouter un second critère de tri (ORDER BY nb_emprunts DESC, num_eleve) le fixe.

> **À RETENIR**
>
> Toute colonne du SELECT qui n'est pas dans une fonction d'agrégation doit figurer dans le GROUP BY.

Filtrer les groupes : HAVING

Élèves ayant emprunté au moins deux livres

```
SELECT num_eleve, COUNT(*) AS nb_emprunts FROM Emprunt GROUP BY num_eleve HAVING COUNT(*)
>= 2;
```

Résultat :

num_eleve	nb_emprunts
101	2
102	2
103	2

WHERE	HAVING
Filtre les lignes, avant le groupement	Filtre les groupes, après le groupement
Ne contient pas de fonction d'agrégation	Contient en général une fonction d'agrégation

WHERE et HAVING ensemble : genres ayant au moins deux livres parus après 1900

```
SELECT genre, COUNT(*) AS nb FROM Livre WHERE annee > 1900 GROUP BY genre HAVING COUNT(*)
>= 2;
```

Résultat :

genre	nb
Poésie	2
Roman	3

Étape par étape : 1) WHERE garde les 6 livres parus après 1900 ; 2) GROUP BY forme 3 groupes : Poésie (2 livres), Roman (3), Théâtre (1) ; 3) HAVING ne garde que les groupes d'au moins 2 livres : Théâtre est éliminé.

6.2 Requêtes sur plusieurs tables : les jointures

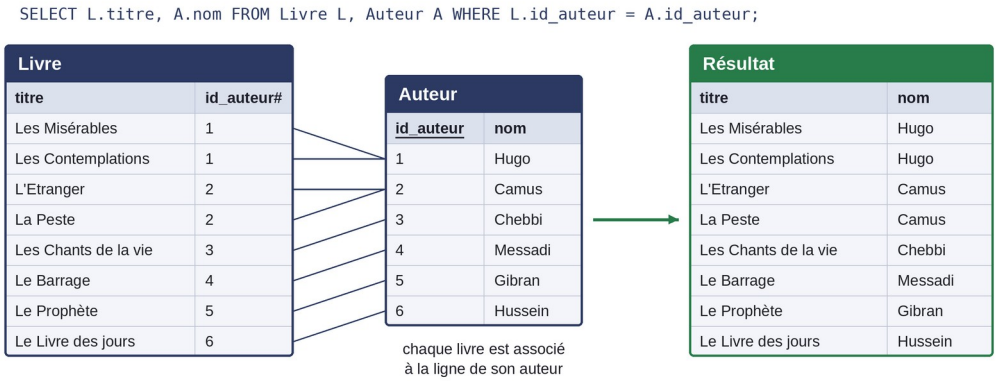
Une jointure combine les lignes de plusieurs tables en reliant la clé étrangère de la table fille à la clé primaire de la table mère.

Méthode pour écrire une jointure : 1) repérer les tables qui contiennent les colonnes demandées ; 2) écrire une condition de jointure par lien entre ces tables : n tables → n − 1 conditions (clé étrangère = clé primaire) ; 3) ajouter les autres conditions avec AND.

Jointure dans la clause WHERE

Titre de chaque livre avec le nom de son auteur

```
SELECT Livre.titre, Auteur.nom FROM Livre, Auteur WHERE Livre.id_auteur = Auteur.id_auteur;
```



Jointure : chaque livre est associé à la ligne de son auteur

Avec des alias de tables, la requête est plus courte :

```
SELECT L.titre, A.nom, A.nationalite FROM Livre L, Auteur A WHERE L.id_auteur = A.id_auteur
AND A.nationalite = 'Tunisie';
```

Résultat :

titre	nom	nationalite
Les Chants de la vie	Chebbi	Tunisie
Le Barrage	Messadi	Tunisie

Explication : L et A sont des alias de tables : L.titre se lit « la colonne titre de la table Livre ». Le préfixe est obligatoire quand deux tables ont une colonne de même nom (ici id_auteur).

ATTENTION

Oublier la condition de jointure associe chaque livre à chaque auteur : 8 × 6 = 48 lignes sans signification (produit cartésien).

```
SELECT COUNT(*) FROM Livre, Auteur;
```

Résultat :

COUNT(*)
48

Explication : sans condition de jointure, chaque livre est combiné avec chaque auteur.

Écriture équivalente avec JOIN ... ON

La même jointure peut s'écrire avec JOIN ... ON ; le résultat est identique :

```
SELECT L.titre, A.nom FROM Livre L JOIN Auteur A ON L.id_auteur = A.id_auteur WHERE
A.nationalite = 'Tunisie';
```

Résultat :

titre	nom
Les Chants de la vie	Chebbi
Le Barrage	Messadi

Jointure de trois tables

Emprunts non rendus : élève, classe et titre

```
SELECT E.nom, E.prenom, E.classe, L.titre, M.date_emprunt FROM Emprunt M, Eleve E, Livre L
WHERE M.num_eleve = E.num_eleve AND M.id_livre = L.id_livre AND M.date_retour IS NULL ORDER
BY M.date_emprunt;
```

Résultat :

nom	prenom	classe	titre	date_emprunt
Ben Salah	Leila	4SI1	L'Etranger	2026-10-01
Jaziri	Omar	4SI2	Le Barrage	2026-10-05
Gharbi	Amel	4SI2	Les Misérables	2026-10-06

Explication : 3 tables → 2 conditions de jointure (Emprunt–Eleve et Emprunt–Livre), plus la condition date_retour IS NULL.

Jointure et groupement

Nombre d'emprunts par classe

```
SELECT E.classe, COUNT(*) AS nb_emprunts FROM Emprunt M, Eleve E WHERE M.num_eleve =
E.num_eleve GROUP BY E.classe;
```

Résultat :

classe	nb_emprunts
3SI1	1
4SI1	4
4SI2	3

Explication : la jointure apporte la classe de chaque emprunteur, puis GROUP BY regroupe les emprunts par classe.

Nationalités ayant au moins deux livres

```
SELECT A.nationalite, COUNT(*) AS nb_livres FROM Livre L, Auteur A WHERE L.id_auteur =
A.id_auteur GROUP BY A.nationalite HAVING COUNT(*) >= 2;
```

Résultat :

nationalite	nb_livres
France	4
Tunisie	2

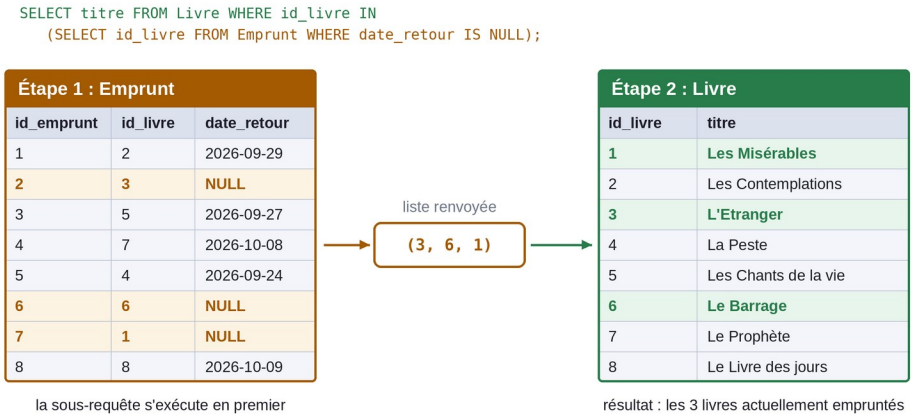
6.3 Requêtes imbriquées

Une requête imbriquée (sous-requête) est un SELECT placé entre parenthèses dans la clause WHERE d'une autre requête. On étudie les sous-requêtes non corrélées : la sous-requête est indépendante, elle s'exécute une seule fois et son résultat sert à la requête principale.

Avec IN et NOT IN : la sous-requête renvoie une liste

Titres des livres actuellement empruntés

```
SELECT titre FROM Livre WHERE id_livre IN (SELECT id_livre FROM Emprunt WHERE date_retour
IS NULL);
```



Sous-requête : la liste renvoyée par la sous-requête filtre la requête principale

Élèves n'ayant jamais emprunté de livre

```
SELECT nom, prenom FROM Eleve WHERE num_eleve NOT IN (SELECT num_eleve FROM Emprunt);
```

Résultat :

nom	prenom
Haddad	Sami

Explication : la sous-requête renvoie la liste des élèves qui ont emprunté (101, 102, 103, 104, 105) ; NOT IN garde les élèves absents de cette liste.

Avec un opérateur de comparaison : la sous-requête renvoie une seule valeur

Livres plus chers que la moyenne

```
SELECT titre, prix FROM Livre WHERE prix > (SELECT AVG(prix) FROM Livre);
```

Résultat :

titre	prix
Les Misérables	25.500
Le Livre des jours	13.500

Explication : la sous-requête calcule d'abord la moyenne, 12.3 ; la requête principale devient alors WHERE prix > 12.3.

Le livre le plus ancien

```
SELECT titre, annee FROM Livre WHERE annee = (SELECT MIN(annee) FROM Livre);
```

Résultat :

titre	annee
Les Contemplations	1856

Sous-requêtes emboîtées

Élèves ayant emprunté un livre d'un auteur tunisien

```
SELECT nom, prenom FROM Eleve WHERE num_eleve IN (SELECT num_eleve FROM Emprunt WHERE
id_livre IN (SELECT id_livre FROM Livre WHERE id_auteur IN (SELECT id_auteur FROM Auteur
WHERE nationalite = 'Tunisie')));
```

Résultat :

nom	prenom
Trabelsi	Karim
Jaziri	Omar

Lire de l'intérieur vers l'extérieur : on lit de la sous-requête la plus intérieure vers l'extérieur : auteurs tunisiens → (3, 4) ; leurs livres → (5, 6) ; élèves qui ont emprunté ces livres → (102, 105) ; noms de ces élèves → Trabelsi Karim, Jaziri Omar.

> **ATTENTION**
>
> Avec =, >, <..., la sous-requête doit renvoyer une seule valeur (souvent une fonction d'agrégation). Si elle peut renvoyer plusieurs lignes, utiliser IN.

Une même question peut souvent se résoudre par une jointure ou par une sous-requête : « livres actuellement empruntés » s'écrit aussi avec une jointure Livre–Emprunt.

```
SELECT L.titre FROM Livre L, Emprunt M
WHERE L.id_livre = M.id_livre AND M.date_retour IS NULL;
```

Résultat :

titre
L'Etranger
Le Barrage
Les Misérables

Explication : même liste de titres que la sous-requête du début de la section, éventuellement dans un autre ordre.

À RETENIR

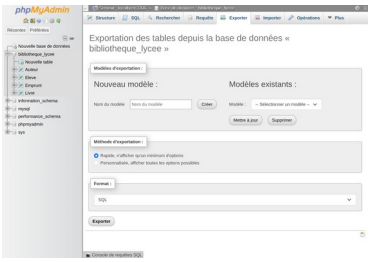
- Mono-table : SELECT … FROM … WHERE … ORDER BY.
- Agrégats : COUNT, SUM, AVG, MIN, MAX ; GROUP BY pour grouper, HAVING pour filtrer les groupes.
- Jointure : FROM table1, table2 WHERE cle_etrangere = cle_primaire.
- Sous-requête non corrélée : IN / NOT IN pour une liste, =, >, < pour une valeur unique.

Annexe — Importer et exporter au format SQL

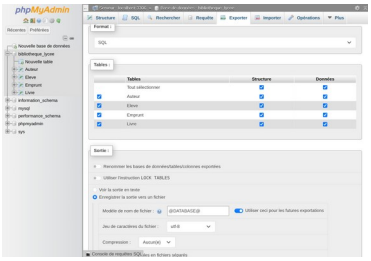
Exporter une base au format SQL, c'est produire un fichier .sql contenant les requêtes qui la recréent : CREATE TABLE pour la structure, INSERT INTO pour les données. Importer, c'est exécuter ce fichier dans le SGBD. Usages : sauvegarde, transfert vers un autre poste, remise d'un travail.

A.1 Exporter avec phpMyAdmin

1. Sélectionner la base (ou une table) dans le panneau de gauche, puis l'onglet Exporter.
2. Méthode « Rapide », format SQL : structure et données de toutes les tables.
3. Méthode « Personnalisée » : choisir les tables et, pour chacune, Structure, Données, ou les deux.
4. Cliquer sur Exporter : le fichier bibliotheque_lycee.sql est téléchargé.



Onglet Exporter de la base : méthode rapide, format SQL



Méthode personnalisée : structure et/ou données, table par table

A.2 Contenu d'un fichier exporté

Structure seule de la table Auteur :

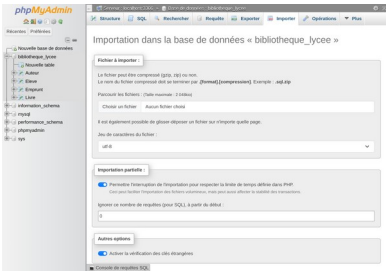
```
CREATE TABLE `Auteur` (
  `id_auteur` int(11) NOT NULL,
  `nom` varchar(50) NOT NULL,
  `prenom` varchar(50) DEFAULT NULL,
  `nationalite` varchar(30) DEFAULT 'Inconnue',
  PRIMARY KEY (`id_auteur`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Données seules de la table Auteur :

```
INSERT INTO `Auteur` VALUES (1,'Hugo','Victor','France');
INSERT INTO `Auteur` VALUES (2,'Camus','Albert','France');
INSERT INTO `Auteur` VALUES (3,'Chebbi','Abou el Kacem','Tunisie');
```

A.3 Importer avec phpMyAdmin

1. Créer la base vide si le fichier ne contient pas CREATE DATABASE, puis la sélectionner.
2. Onglet Importer : « Choisir un fichier », sélectionner le fichier .sql.
3. Cliquer sur Importer : phpMyAdmin exécute les requêtes et affiche leur nombre.



Onglet Importer de la base

On peut aussi coller le contenu du fichier dans l'onglet SQL et l'exécuter : c'est exactement ce que fait l'importation.

Mémo SQL

Définition des données	Rôle
CREATE DATABASE nom / DROP DATABASE nom	Crée / supprime une base
CREATE TABLE t (col type [contrainte], …, [CONSTRAINT nom] contrainte)	Crée une table
ALTER TABLE t ADD col type [contrainte]	Ajoute une colonne
ALTER TABLE t DROP col	Supprime une colonne
ALTER TABLE t MODIFY col type	Modifie le type (standard : ALTER col type, refusé par XAMPP)
ALTER TABLE t RENAME COLUMN nom1 TO nom2 / CHANGE nom1 nom2 type	Renomme une colonne
ALTER TABLE t ADD CONSTRAINT nom contrainte	Ajoute une contrainte
ALTER TABLE t DROP CONSTRAINT nom	Supprime une contrainte
ALTER TABLE t ENABLE / DISABLE CONSTRAINT nom	Active / désactive (XAMPP : SET FOREIGN_KEY_CHECKS = 1 / 0)
DROP TABLE t	Supprime une table

Manipulation et interrogation	Rôle
INSERT INTO t [(colonnes)] VALUES (valeurs)	Insère des lignes
UPDATE t SET col = expression [WHERE condition]	Modifie des valeurs
DELETE FROM t [WHERE condition]	Supprime des lignes
SELECT [DISTINCT] … FROM … [WHERE] [GROUP BY] [HAVING] [ORDER BY]	Interroge

Opérateurs et fonctions	Liste
Comparaison	= <> > < >= <= IN BETWEEN LIKE IS (NULL)
Logiques	AND OR NOT
Agrégation	AVG COUNT MAX MIN SUM
Dates	DATE() DAY() MONTH() YEAR() NOW() ADDDATE() DATEDIFF()
Types	INT DECIMAL CHAR VARCHAR TEXT DATE TIME DATETIME
Contraintes	NOT NULL DEFAULT CHECK PRIMARY KEY UNIQUE FOREIGN KEY REFERENCES ON UPDATE CASCADE ON DELETE CASCADE